AI TRAVEL OPERATING SYSTEM

PROJECT BLUEPRINT

A production-minded multi-agent travel platform powered by Gemini, ADK and MCP.

Features • Use cases • Agent roles • Architecture • Roadmap

The idea

Build a travel platform where a user describes a trip in natural language and a coordinated team of AI agents researches options, compares providers, creates an itinerary, manages a budget and guides the user through booking—while reliable backend services retain control of prices, availability, payments and reservations.

Core principle: AI plans and coordinates; trusted provider APIs and application services verify facts and perform transactions.

Intended scope

A realistic learning and portfolio project using simulated bookings first, then sandbox/provider integrations. It is not a travel agency or live booking engine until commercial provider agreements and production controls are in place.

1. Product vision

Working name: Wayfinder AI (name is provisional).

Wayfinder AI is an AI-powered travel operating system for planning and coordinating trips across flights, accommodation, ground transport, activities and travel documents. Instead of asking users to search multiple websites and manually combine results, the platform turns a goal into a structured, editable trip plan.

Product goals

- Convert natural-language travel requests into clear trip requirements.
- Search multiple providers through a unified internal API instead of coupling the user interface to individual suppliers.
- Compare options using total trip cost, timing, baggage, cancellation terms, location and user preferences.
- Generate practical day-by-day itineraries with maps, travel time estimates and booking links.
- Keep trip details, decisions, quotes and booking status in one workspace.
- Use agents for research and coordination while deterministic services enforce policy and execute approved actions.

Primary users

User segment	Need	Value delivered
Leisure travellers	Plan a holiday without juggling tabs and spreadsheets.	A coherent trip plan, comparisons and itinerary.
Families and groups	Coordinate dates, budgets, room needs and shared choices.	Shared trip workspace and preference-aware recommendations.
Frequent travellers	Manage multiple trips and changing plans.	Reusable preferences, trip history and disruption assistance.
Travel advisors / small agencies	Prepare proposals and compare suppliers faster.	Agent-assisted research and client-ready itineraries.

2. Core features

Feature	What it does
Conversational trip brief	Collect destination, dates, origin, travellers, budget, interests, pace, accessibility needs and constraints. Ask targeted follow-up questions when essential information is missing.
Flight discovery and comparison	Search connected flight sources; normalize itineraries, stops, duration, baggage, fare rules and price currency; rank by user priorities.
Accommodation discovery	Compare hotels and stays by location, room type, guest capacity, amenities, ratings where available, cancellation rules and total price.
Itinerary builder	Create a day-by-day plan with places, time blocks, estimated travel time, opening-hour checks and alternative activities.
Budget planner	Estimate flights, stays, local transport, activities, food and contingency; show assumptions and remaining budget.
Ground transport and activities	Find airport transfers, car rental, rail/bus options, tours and attractions where provider data is available.
Booking workspace	Save shortlisted options, compare them, create booking drafts, open provider checkout or execute sandbox bookings after confirmation.

Feature	What it does
Trip hub	Store itinerary, booking references, documents, travellers, reminders, notes and change history in one place.
Disruption assistant	Help users understand delays, cancellations or schedule changes and present rebooking options; do not claim to change a booking unless provider confirmation is received.
Collaborative planning	Invite companions, collect votes, compare preferences and maintain one agreed itinerary.
Travel knowledge assistant	Answer destination questions using grounded sources, with source links and clear uncertainty.
Notifications and reminders	Send configurable reminders for check-in, departure, document deadlines and itinerary events.

3. Agent team and responsibilities

Use a small number of clearly bounded agents. Each agent should have a narrow responsibility, typed inputs and outputs, explicit tool permissions and observable execution traces.

1. Travel Coordinator Agent

Role: Main orchestrator. Interprets the request, identifies missing information, creates a trip-planning workflow, delegates tasks, tracks progress and synthesizes the final answer.

Inputs: Trip brief, traveller preferences, task results, trip state.

Tools: Delegate to flight, stay, itinerary, budget and destination agents; ask the user for missing essentials.

Guardrail: Must not invent availability, prices or booking confirmations.

2. Flight Search Agent

Role: Finds and compares flight options from connected suppliers.

Inputs: Origin, destination, dates, passenger count, cabin, baggage and flexibility.

Tools: Search flight provider adapters; normalize results; filter and rank options.

Guardrail: Read/search only in initial release. Booking requires separate confirmation and provider support.

3. Accommodation Agent

Role: Finds stays that match location, budget, room and traveller requirements.

Inputs: Destination, check-in/out, guests, rooms, preferences.

Tools: Search accommodation providers; normalize property, room, cancellation and price data.

Guardrail: Clearly distinguish estimated totals from confirmed prices and availability.

4. Itinerary Agent

Role: Builds a realistic daily plan around selected locations and user interests.

Inputs: Dates, hotel area, interests, pace, mobility constraints, saved places.

Tools: Place search, maps/routing, opening-hours sources and itinerary tools.

Guardrail: Do not fabricate opening hours or travel times; flag unverified details.

5. Budget & Optimization Agent

Role: Builds a cost model and finds trade-offs that fit the budget.

Inputs: Budget, currency, trip duration, flight/stay/activity candidates.

Tools: Currency conversion, cost calculator, ranking and scenario tools.

Guardrail: Show assumptions, date/time of quote and excluded costs.

6. Destination Research Agent

Role: Researches local context such as seasons, neighbourhoods, transport, customs and practical travel considerations.

Inputs: Destination, dates, traveller profile and questions.

Tools: Approved web/search sources and curated destination knowledge.

Guardrail: Cite sources for current claims; distinguish advice from official requirements.

7. Booking & Trip Operations Agent

Role: Coordinates booking drafts, user approvals, provider checkout and trip status updates.

Inputs: Selected offer, traveller details, price quote and approval state.

Tools: Create booking draft, validate quote, initiate provider checkout, retrieve booking status.

Guardrail: Never purchase, cancel or modify without explicit user approval and provider-side verification.

8. Travel Support Agent

Role: Helps with changes, cancellations, itinerary questions and trip support.

Inputs: Trip ID, booking status, user issue and provider policy.

Tools: Retrieve booking details, policy lookup, support case creation and notification tools.

Guardrail: Do not promise refunds or changes until the provider confirms them.

Supporting components (not necessarily agents)

- **Policy and permission service:** deterministic checks for user permissions, price changes, approval requirements and booking actions.
- **Provider adapters:** normal application code that connects to supplier APIs and maps them into a common schema.
- **Notification worker:** sends email, push or in-app reminders from confirmed events.
- **Evaluation pipeline:** scores task completion, factual grounding, tool selection, policy compliance and user satisfaction.

4. Example end-to-end use cases

Use case A — Plan a complete trip

User: “Plan a 7-day trip from Mumbai to Japan in April for two people, under $2,000 each. We like food, culture and anime.”

- Coordinator collects exact dates, passport/nationality only if needed for official entry research, and flexibility.
- Flight and accommodation agents search provider sources and return normalized options.
- Destination agent researches seasonal context and neighbourhoods.
- Itinerary agent drafts a day-by-day plan around selected accommodation areas.
- Budget agent estimates total cost, highlights assumptions and proposes trade-offs.
- Coordinator presents a shortlist with source links and asks the user to choose.

Expected result: A sourced, editable trip proposal—not an unverified claim that everything is booked.

Use case B — Find the best flight

User: “Find the cheapest reasonable flight to London next Friday, but avoid overnight layovers.”

- Coordinator confirms origin airport and whether nearby airports are acceptable.
- Flight agent queries available suppliers and normalizes fare, baggage, stops and duration.
- Budget agent compares total cost and trade-offs.
- Coordinator presents ranked options with quote timestamps and provider links.

Expected result: A transparent comparison that explains why each option ranked where it did.

Use case C — Optimize an existing itinerary

User: “Move the museum visit to the morning and group nearby attractions together.”

- Itinerary agent retrieves saved itinerary and geospatial coordinates.
- Itinerary agent reorders stops while checking opening hours and travel-time data.
- Coordinator checks for conflicts with confirmed bookings and meal/rest blocks.
- Updated itinerary is shown for user approval and saved with a version history.

Expected result: A revised itinerary that avoids conflicts and preserves confirmed reservations.

Use case D — Booking with explicit approval

User: “Book this hotel room.”

- Booking agent retrieves the selected offer and requests a fresh price/availability check.
- Backend validates the offer, cancellation terms, traveller details and total payable amount.
- System displays provider, dates, room, policy, currency and final total.
- User explicitly confirms the exact booking.
- Backend initiates provider checkout or sandbox booking using an idempotency key.
- System reports success only after receiving and verifying provider confirmation.

Expected result: A controlled booking workflow with an auditable approval and provider confirmation.

Use case E — Disruption support

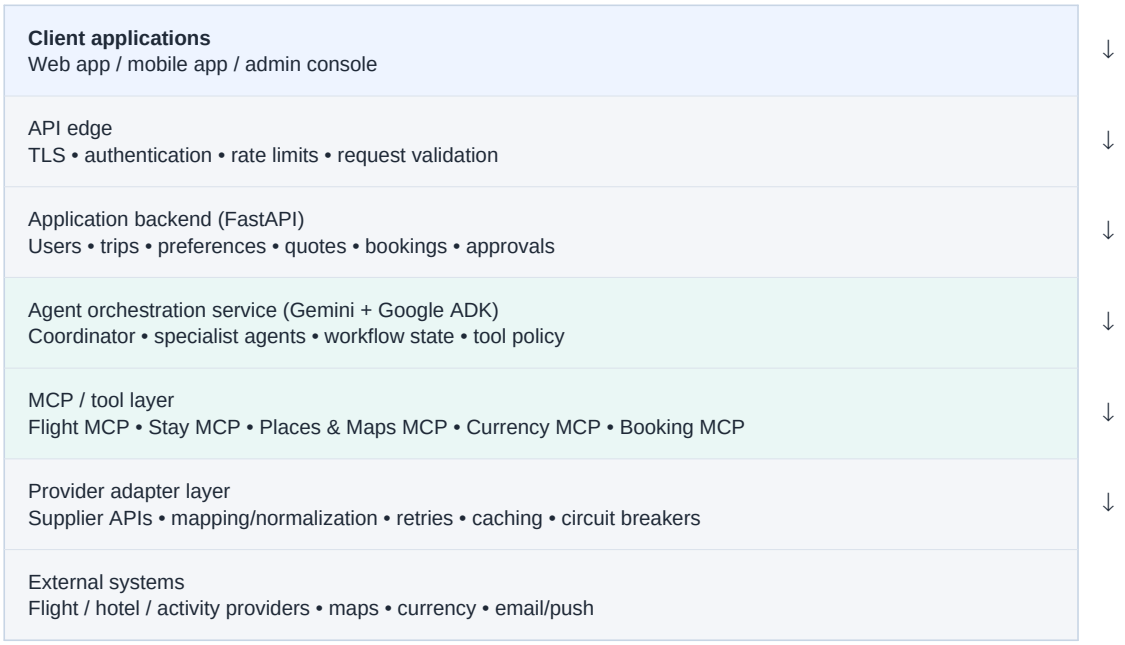
User: “My flight time changed. Can you adjust my plan?”

- Support agent retrieves the latest booking and flight status from connected sources.
- Coordinator identifies affected itinerary events and ground transport.
- Itinerary agent proposes a revised plan and highlights conflicts.
- If a flight change or cancellation is needed, present provider options and terms.
- User approves any paid or irreversible action; provider confirmation updates trip status.

Expected result: A revised plan with clear separation between suggestions and actions actually completed.

5. High-level system architecture

Recommended architecture: a modular application with a dedicated agent service, provider integration layer and durable trip/booking backend. Begin as a modular monolith plus independently deployable MCP servers; split services further only when operational needs justify it.



Core data stores

Store	Purpose
PostgreSQL	Users, travellers, trips, itinerary versions, normalized offers, booking records, approvals and audit events.
Redis	Short-lived cache, rate limiting, locks, task coordination and transient session data.
Object storage	Travel documents and generated exports, protected by private access and retention rules.
Search / vector index (optional)	Destination knowledge retrieval and semantic search over curated travel content.

Important separation of responsibilities

- Gemini/agents decide what information or tool is needed; they do not own the source of truth.
- Provider adapters own supplier communication, schema normalization and provider-specific error handling.
- The backend owns users, permissions, quotes, trip state, approvals and booking records.
- The provider is authoritative for live availability, final price and booking confirmation.
- MCP exposes selected capabilities to agents; it is not a replacement for REST APIs, business logic or database authorization.

6. Suggested technology stack

Layer	Recommended choice	Why
LLM	Gemini API	Natural-language understanding, structured outputs and tool/function calling.

Layer	Recommended choice	Why
Agent framework	Google ADK	Agent definitions, delegation, workflows, sessions and evaluation support.
Backend	Python + FastAPI + Pydantic	Typed APIs, validation and a strong Python ecosystem.
MCP	MCP Python SDK	Standardized tool exposure and agent-to-tool connectivity.
Database	PostgreSQL + SQLAlchemy	Relational trip, offer, booking and audit data.
Cache / jobs	Redis + task queue	Caching, rate limits and asynchronous work.
Frontend	React / Next.js; optional Flutter	Responsive trip workspace and mobile experience.
Maps	A mapping/routing provider	Geocoding, distance, routes and place details.
Deployment	Docker + Google Cloud Run	Containerized services and managed deployment.
Observability	OpenTelemetry + metrics/log platform	Trace agent runs, API calls, latency and failures.
CI/CD	GitHub Actions	Automated tests, security checks and deployments.

7. Unified provider API strategy

The frontend and agents should not integrate separately with every flight or hotel supplier. Build a provider abstraction layer that translates supplier-specific APIs into your own stable domain model.

Example flight search contract

```
POST /v1/flights/search { "origin": "BOM", "destination": "NRT", "departure_date": "2027-04-12", "return_date": "2027-04-19", "adults": 2, "cabin": "economy", "currency": "USD" }
```

Normalized response (illustrative schema)

```
{ "search_id": "srch_...", "offers": [{ "offer_id": "off_...", "provider": "supplier_name", "itinerary": [], "price": { "amount": 840.00, "currency": "USD", "price_type": "quoted" }, "baggage": {}, "fare_rules": {}, "last_verified_at": "timestamp", "booking_url": "provider_checkout_url" }] }
```

Use a common schema for flights, accommodation, activities and transport. Preserve provider IDs and raw supplier references for traceability. Never silently merge different fare conditions or cancellation policies.

Provider integration considerations

- Use official APIs, affiliate programs or approved partner channels. Access, commercial terms and booking rights vary by provider.
- Some providers expose search but not booking; others require affiliate approval, certification or specific checkout flows.
- Do not scrape sites or assume that a public consumer website provides a usable production API.
- Keep supplier credentials server-side; use secret management and rotate credentials.
- Store quote timestamps and expiry where supplied. Revalidate price and availability before checkout.
- Design provider adapters so a supplier can be added or replaced without changing agent prompts or frontend contracts.

8. Booking, trust and safety design

Travel decisions can involve money, identity information and non-refundable purchases. Treat booking as a high-impact workflow with deterministic checks and explicit consent.

Control	Required behaviour
Authentication	Authenticate users and protect trip data by account and role.
Authorization	Check access in backend services for every read/write; never rely on the agent prompt.
Price confirmation	Recheck live price, taxes, currency and availability immediately before purchase.
Explicit approval	Show exact provider, traveller, dates, product, cancellation policy and total; require an affirmative user action.
Idempotency	Use idempotency keys and transaction records to avoid duplicate booking attempts.
Auditability	Record who requested, approved and executed each action, with timestamps and provider references.
Privacy	Minimize passport and personal data; encrypt sensitive fields and apply retention controls.
Reliability	Use timeouts, bounded retries, circuit breakers and reconciliation for uncertain provider outcomes.
Honest status	Say “request submitted” until provider confirmation is verified; never invent a booking reference.

Booking state machine

DRAFT → QUOTE_VALIDATED → AWAITING_USER_APPROVAL → SUBMITTED → CONFIRMED / FAILED / UNKNOWN → RECONCILIATION

If a provider times out after submission, the outcome may be UNKNOWN. Query the provider using its reference or idempotency key before retrying; do not blindly submit a second booking.

9. MVP scope and phased roadmap

Phase	Build	Exit criteria
0. Discovery	Define personas, core user journeys, provider access assumptions and non-goals.	A short product spec, architecture decision record and test scenarios.
1. Trip workspace	Authentication, profiles, preferences, trips, saved options and itinerary CRUD.	A user can create and edit a trip without AI.
2. Provider foundation	One flight search source, one accommodation source, maps/places and a unified schema.	Search results are normalized, traceable and timestamped.
3. First agents	Coordinator, flight, stay, itinerary and budget agents using Gemini/ADK.	A natural-language request produces a sourced trip proposal.
4. MCP layer	Expose selected provider and application tools through MCP servers.	Agents can call tools with typed schemas and permission checks.
5. Booking simulation	Draft, validate, approve and confirm mock bookings in a sandbox ledger.	No real money; all state transitions and approvals are auditable.
6. Production hardening	Observability, evaluation, security tests, rate limits, retries, load tests and CI/CD.	Operational dashboards, measurable quality and recovery procedures.
7. Controlled live pilot	Add approved supplier booking/checkout flows and support workflows.	Provider contracts, security review, reconciliation and support procedures are ready.

Suggested MVP: what to build first

- Trip creation through chat with clarification for missing dates, origin and traveller count.
- Search and compare flights from one approved source.
- Search and compare stays from one approved source.
- Generate a 3–7 day itinerary using places and routing data.
- Create a budget estimate with visible assumptions.
- Save options and itinerary versions to a user account.
- Use simulated booking confirmation before attempting any real booking integration.

Avoid in the first release

- Autonomous purchases, cancellations or non-refundable changes.
- Dozens of providers before the unified schema is stable.
- Too many agents with overlapping responsibilities.
- Unverified visa/immigration advice or claims of guaranteed entry.
- Complex loyalty points, insurance underwriting or dynamic package fulfilment.

10. Evaluation and production-readiness

Create a repeatable evaluation set rather than judging the system by a few successful demos.

Area	What to measure / test
Planning quality	Does the plan satisfy dates, budget, interests, pace and traveller constraints?
Grounding	Are prices, availability, policies and destination claims tied to current source data?

Area	What to measure / test
Tool correctness	Did the agent select the right tool, pass valid parameters and handle errors?
Workflow reliability	Can it resume after a timeout, avoid duplicate actions and preserve trip state?
Safety	Can it access another user's trip, expose secrets or book without approval? Expected answer: no.
User experience	How often does it ask unnecessary questions? Are comparisons understandable and useful?
Cost and latency	Track model calls, tokens, provider calls, retries, end-to-end latency and cost per completed trip.

13. Example MCP tool catalogue

Expose narrowly scoped tools. Use structured schemas, strict validation and authorization at the service boundary.

MCP server	Example tools
Flight tools	search_flights, get_offer_details, refresh_flight_quote
Accommodation tools	search_stays, get_property_details, refresh_stay_quote
Places & maps tools	search_places, get_place_details, get_route_matrix, geocode_location
Trip tools	create_trip, get_trip, save_itinerary_version, shortlist_offer
Booking tools	create_booking_draft, validate_booking, request_user_approval, submit_approved_booking, get_booking_status
Currency tools	get_exchange_rate, convert_amount

14. Key product decisions to make early

- **Market:** Start with one geography or a small set of routes/destinations to keep provider coverage manageable.
- **Business model:** Affiliate referrals, subscription trip planning, agency tooling or a hybrid model. Do not assume every supplier supports affiliate booking.
- **Booking model:** Initially redirect to supplier checkout; later add approved booking APIs where commercially and technically supported.
- **Data freshness:** Define quote expiry, refresh rules and how stale results are displayed.
- **Agent autonomy:** Allow autonomous research and planning; require user approval for purchases, cancellations and consequential changes.
- **Source transparency:** Show supplier, source link, retrieval time, price currency and what is excluded.

15. Definition of done

- A user can create a trip through a conversational interface and edit the structured trip brief.
- At least one live/approved flight source and one stay source are integrated through provider adapters—or clearly labelled mock providers in development.
- Agents collaborate to create a sourced itinerary and budget estimate.
- All tool calls are typed, authorized, logged and traceable to a user/session.
- No booking can be submitted without fresh validation and explicit user approval.
- The system handles provider errors, timeouts, stale quotes and duplicate requests safely.

- Automated tests cover core APIs, agent workflows, permissions and booking state transitions.
- The application is containerized, deployed, monitored and documented.

Recommended build order

Backend and unified provider schema → trip workspace → one flight and one stay integration → coordinator and specialist agents → MCP tools → itinerary and budget → simulated booking approvals → observability and evaluation → controlled live provider workflows.

This blueprint is a technical learning plan, not a guarantee of supplier API access or a substitute for provider contracts, security review, legal review or operational readiness. Provider capabilities and commercial terms must be verified before production use.